Terraform Modules: Beginner Guide to Reusable Code

Stop copying and pasting infrastructure blocks across your dev, staging, and production environments. Terraform modules let you package cloud resources into clean, reusable building blocks that save you hours of work.

What you'll learn:

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

1. Building a Local Module Structure

A Terraform module is simply a folder containing .tf files. Keep your main resource definitions in main.tf, accept configurable settings in variables.tf, and expose useful results in outputs.tf. You can then reference this folder from anywhere in your root configuration using the source argument.

```
module "app_vpc" {
  source      = "./modules/vpc"
  cidr_block  = "10.0.0.0/16"
  environment = "production"
}
```

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

2. Passing Inputs and Exposing Outputs

Think of variables as module parameters and outputs as return values. When your module creates an AWS resource like a database, export its endpoint in outputs.tf. Your root configuration or other downstream modules can immediately read that output without hardcoding IP addresses.

```
# Inside root main.tf
module "db" {
  source = "./modules/rds"
}

module "web" {
  source = "./modules/ec2"
  db_url = module.db.endpoint # Reads output from db module
}
```

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

3. Using Pre-Built Registry Modules

The official Terraform Registry hosts thousands of battle-tested modules maintained by cloud providers. Instead of writing 200 lines of raw Terraform to configure an AWS VPC with public subnets, private subnets, and internet gateways, you can pull the official community module with standard settings.

```
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.1.2"

  name = "main-vpc"
  cidr = "10.0.0.0/16"
  azs  = ["us-east-1a", "us-east-1b"]
}
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

4. Downloading and Updating Modules

Whenever you add, modify, or upgrade a module source in your Terraform files, you must run terraform init. Terraform downloads the remote module code into a hidden local cache directory (.terraform/modules). If you bump the module version, run init with the upgrade flag.

```
$# Download all modules
 terraform init

 # Upgrade modules to latest allowed versions
 terraform init -upgrade
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

- + Always specify an explicit version argument when calling registry modules to avoid breaking changes.
- + Keep module folders self-contained with their own README.md, variables.tf, and outputs.tf.
- + Use meaningful variable descriptions and types (e.g. type = string) to help teammates understand inputs.
- + Avoid writing giant 'monolithic' modules—break them into focused pieces like networking, compute, and database.
- + Provide sensible default values for optional variables so callers only need to provide mandatory fields.

Watch Out For

- ! Hardcoding resource names and environment strings inside module files instead of using variables.
- ! Forgetting to run 'terraform init' after adding a new module or changing its source path.
- ! Creating tight circular dependencies where Module A depends on Module B and Module B depends on Module A.
- ! Writing modules that are too specific to one single project rather than genuinely reusable across teams.